

Final Report

COMPENG 3DY4 - Group 70

Azaan Khan (400399089)

Ayaan Hussain (400540174)

Haseeb Shaikh (400521659)

Taimur Ahmed (400514463)

April 2nd, 2026

Introduction

The goal of this project was to design and implement a real-time software-defined radio receiver that could process FM broadcast signals and recover mono audio, stereo audio, and RDS data. From an application standpoint, the project was about building a receiver that could take in real FM radio signals and output useful audio and station information, rather than just studying individual DSP blocks in isolation. From a practical standpoint, the software had to run in real time on a constrained hardware setup using an RTL-SDR front end and a Raspberry Pi, with raw 8-bit interleaved I/Q samples entering the program through standard input and processed audio written to standard output for playback. This meant the project involved not only DSP ideas such as filtering, demodulation, resampling, and decoding, but also implementation concerns like block-based processing, multithreading, queue communication, and performance optimization needed to keep the system running reliably in real time.

Project Overview

This project involves building a real-time FM receiver in software using a software-defined radio (SDR) approach. Instead of relying on dedicated hardware blocks for each stage of radio reception, the system processes raw I/Q samples in software to recover mono audio, stereo audio, and radio data system (RDS) information from an FM broadcast signal. This makes the project a practical combination of digital signal processing, communications, and embedded systems.

At a high level, the receiver is organized as a signal-flow graph made up of standard DSP blocks. The RF front end first selects the desired FM channel, reduces the sample rate, and performs FM demodulation. The demodulated signal is then used by the mono, stereo, and RDS paths. The mono path extracts the main audio band, the stereo path recovers the stereo information using the 19 kHz pilot tone and carrier recovery, and the RDS path extracts the digital data channel centred around 57 kHz.

Several basic building blocks are used throughout the receiver, including finite impulse response (FIR) filters, FM demodulators, phase-locked loops (PLLs), decimators, and resamplers. FIR filters are used to isolate the required frequency bands, while sample-rate conversion is used to move signals to rates that are more suitable for later stages of processing. In the stereo path, a PLL is used to lock onto the pilot tone and regenerate the carrier needed for demodulation. In the RDS path, additional digital communication steps such as matched filtering, timing recovery, Manchester decoding, differential decoding, and frame synchronization are needed to recover the transmitted bitstream and extract useful station information.

Thus, the project shows how filtering, demodulation, synchronization, resampling, and decoding can be chained together to implement a complete FM receiver in software. This big-picture background provides the context for the implementation choices and design steps discussed in the next section.

The receiver was designed to support multiple operating modes with different sample-rate settings, which changes the required rate conversion and processing load across the mono, stereo, and RDS paths.

Parameter	Mode 0	Mode 1	Mode 2	Mode 3
RF F_s (kSamples/sec)	2400	2304	2400	1800

IF Fs (Ksamples/sec)	240	288	240	360
Audio Fs (KSamples/sec)	48	36	44.1	44.1
Block duration (ms)	40	56	60	30
RF Decimation Factor	10	8	10	5
Audio Decimation D	5	8	800	400
Audio Upsample Factor U	1	1	147	49

Table 1: Sample rate parameters for Group 70.

The processing path is selected at runtime: m for mono, s for stereo, r for RDS. When the RDS path is selected, stereo audio is simultaneously produced. RDS is supported only in modes 0 and 2, where IF Fs = 240 kHz, satisfying the Nyquist criterion for the 114 kHz tone used in RDS carrier recovery. The number of filter taps defaults to 101 and can be overridden by a third command-line argument, allowing the tap count to be reduced when operating at lower CPU frequencies as specified in Table 2.

CPU Frequency Range	Filter Taps
600 MHz to 900 MHz	81
900 MHz to 1200 MHz	87
1200 MHz to 1500 MHz	95
1500 MHz and above	101

Table 2. Filter tap count requirements per CPU frequency range.

Settings for group 70	Mode 0	Mode 2
Samples per symbol (SPS)	42	21

Table 3. Group 70 custom RDS samples-per-symbol (SPS) settings for modes 0 and 2, used for clock and data recovery at the RDS-path input sample rate of 240 kSamples/sec.

Implementation Details

Our implementation developed gradually from the earlier labs into the final integrated SDR system. We definitely did not approach the project by trying to build everything at once. The labs gave us a solid starting point, since we were already familiar with things like FIR filter design, convolution-based filtering, decimation, and FM demodulation. From there, we built outward

into the mono, stereo, and RDS paths. A big part of our workflow was using Python first to model new sections and make sure the DSP made sense before moving them into C++. That helped a lot with debugging, because it gave us something to compare against instead of trying to figure everything out directly in the full C++ system.

The first part we really needed to get working was the RF front end, since everything after that depends on it. The RTL-SDR provides interleaved 8-bit unsigned I/Q data, so our implementation began by reading the raw samples, normalizing them into real-valued form, and separating them into I and Q streams. After that, each stream was low-pass filtered and decimated to the intermediate-frequency sample rate, and the filtered I/Q streams were then FM-demodulated to produce the real-valued signal that feeds the mono, stereo, and RDS branches. Conceptually, this stage was close to what we had already done in the labs, but one of the first implementation issues we encountered was that the project had to work in a block-based real-time style rather than over one full signal stored in memory. Because of that, we had to preserve filter state between blocks. This was one of the first places where we had to distinguish between a conceptual DSP problem and an implementation problem: the filtering idea itself was already known, but representing it correctly in C++ with block continuity required extra care.

Once the RF front end was producing a reasonable FM-demodulated signal, we moved to the mono path. We treated mono as the baseline checkpoint for the whole project because it was the simplest complete end-to-end output path and gave us an immediate way to test whether the earlier parts of the chain were working properly. In the mono path, we extracted the 0 to 15 kHz audio content from the FM-demodulated signal using low-pass filtering and then converted it to the required audio output rate. In mode 0, this was relatively simple, since the IF rate was 240 kSamples/sec and the audio output rate was 48 kSamples/sec, so we could use integer decimation by 5 after filtering. Getting mono audio working was important because it gave us confirmation that the front-end filtering, decimation, FM demodulation, and audio formatting were all consistent enough to produce something usable.

After mono was working, we extended the design to stereo. This was the point where the project started to feel a lot more complex than the labs. For stereo, the receiver does not directly get separate left and right channels. Instead, it gets the mono signal $L+R$, a 19 kHz pilot tone, and the stereo difference signal $L-R$ around a suppressed 38 kHz subcarrier. Because of that, the stereo path needed several separate stages. We had to isolate the 19 kHz pilot tone with a band-pass filter, use a PLL and NCO to regenerate the 38 kHz carrier, extract the stereo band itself, and then mix that signal back down to baseband. After that, the recovered $L-R$ signal could be low-pass filtered, converted to the audio rate, and combined with $L+R$ to rebuild the left and right channels.

One issue that came up in the stereo path was that the mono and stereo branches do not naturally have the same delay. Since the stereo branch goes through extra filtering stages, the two signals can end up slightly misaligned if they are recombined directly. In practice, we treated this mono-path delay block like an all-pass delay stage whose purpose was simply to align the two branches without changing the useful mono content. One useful thing we realized is that this delay block does not have to be placed in only one exact spot in the mono path. For example, it could be placed after the mono low-pass filter if that made the implementation cleaner, as long as the total delay was still correct by the time the mono and stereo signals were combined. So the main thing that matters is not the exact location of the block, but whether the two branches line up properly at the stereo combiner.

The RDS path was the most challenging section of the project and the part that pushed us furthest beyond the earlier labs. After extracting the RDS band, we had to recover the carrier and downconvert the RDS signal to baseband. However, even after doing that, the signal was still not sampled at a rate that made symbol timing recovery easy. Because of this, we used rational resampling so that the baseband RDS signal would end up with an integer number of samples per symbol. As a group, we targeted a convenient samples-per-symbol value so that later timing recovery could search over a fixed number of possible offsets instead of dealing with an awkward fractional relation between sampling rate and symbol rate.

A major part of our RDS modelling work involved understanding the mathematics behind the Python resampling stage, especially the use of `signal.upfirdn`. At first, it is easy to think of `signal.upfirdn` as just a built-in processing step, but we had to understand what it was actually doing before we could explain it properly or reimplement it ourselves. If a discrete-time signal $x[n]$ is sampled at a rate F_s , and we want a new rate of F_s times U over D , then the resampler can be understood as three stages: upsampling by U , FIR filtering, and downsampling by D . Upsampling by U means inserting $U-1$ zeros between each original sample, which raises the sampling rate to UF_s . This creates imaging artifacts, so a low-pass filter is needed afterward as an anti-imaging filter. Finally, downsampling by D keeps only every D th sample, giving the final rate change of U divided by D . A simple way to write the full process is:

$$y[n] = \text{downsample by } D \text{ of } (h[n] \text{ convolved with upsample by } U \text{ of } x[n])$$

This is exactly what `signal.upfirdn` does in one step: it upsamples, applies the FIR filter, and then downsamples. For us, this mattered because simple integer decimation was not enough in the RDS path. We needed a correct sample-rate conversion so the baseband RDS signal could be moved to a rate with a convenient number of samples per symbol, which then made clock and data recovery much easier.

After resampling, we applied a matched filter in the form of an RRC filter. This stage was important because the RDS signal is relatively weak and noisy compared to the mono and stereo content, so the waveform benefits from being shaped in a way that improves symbol decisions. In simple terms, the matched filter helped make the symbol peaks more reliable and reduce the effect of intersymbol interference before clock recovery. From there, our RDS implementation moved into clock and data recovery. We used a warmup approach in which several matched-filtered blocks were accumulated, candidate symbol offsets across one symbol period were tested, and the best offset was selected. We also compared the I and Q decision axes and chose whichever one gave cleaner separation for hard decisions. After that, the steady-state symbol sampler could simply sample once per symbol using the chosen phase and slice the result to $+1$ or -1 .

After CDR, the remaining stages of the RDS path shifted more into communications and data decoding. Since RDS uses Manchester coding, the hard-sliced symbols could not be treated directly as bits. Instead, we had to examine symbol pairs and decode them according to their transitions. After Manchester decoding, we applied differential decoding to handle polarity ambiguity that can arise from carrier recovery. The resulting bit stream was then searched for valid 26-bit RDS blocks using syndrome checking and known offset words. Each block contains 16 information bits and 10 check bits, so once valid block alignment was established, we could extract useful application-layer fields like PI code, group type, PTY, PS name, and RadioText.

Throughout the project, our validation strategy was to move in small steps and compare against expected intermediate behaviour whenever possible. For mono and stereo, that often meant

checking spectra, listening to audio output, and comparing behaviour against the Python models. For RDS, it meant verifying whether the expected sample-rate conversion, matched-filtered waveforms, CDR output, and decoding stages were behaving consistently before moving further down the chain. This iterative approach was especially useful because the implementation problems we faced were not all of the same type. Some were conceptual, such as choosing the wrong filter settings, using an inconvenient resampling ratio, or seeing poor stereo behaviour due to branch-delay mismatch. Others were implementation-specific, such as incorrect array or vector sizing, indexing errors, block-boundary mistakes, state-handling problems, or queue/thread synchronization issues.

Optimization was another major part of the final implementation, since a straightforward implementation of everything would be too expensive for sustained real-time use. Our group used block-based filtering with preserved state, fast decimation functions that avoided computing outputs that would later be discarded, and fast rational resampling rather than explicitly constructing a large zero-stuffed intermediate stream. We also structured the system using multiple threads and queues so that RF processing, audio output, and RDS processing could proceed concurrently. This was necessary to keep the system closer to real-time behavior and to prevent one part of the chain from blocking the others.

Overall, our implementation progressed from simpler and more observable stages to more complex and more sensitive ones. As a group, we used mono as the first working baseline, extended that into stereo with carrier recovery and branch alignment, and then moved into the RDS path with rational resampling, matched filtering, timing recovery, and decoding. Across all of these stages, the most effective strategy for us was to model new blocks first, validate them as much as possible, and only then translate them into the C++ real-time structure. That approach made the project much more manageable and also gave us a clearer understanding of which difficulties came from the underlying signal-processing theory and which came from implementation details in code.

Haseeb's Interview Response: To move the all-pass delay in the mono path to a point after the low-pass filter, the condition that must be satisfied is that the delay for the all-pass filter must be applied at the same sample rate at which the stereo path's bandpass filters operate, so that the phase delay on the mono path and stereo path are identical when the left and right channels are recombined. In mode 0, the mono path starts at the IF sample rate of 240 kSamples/sec and is then decimated by 5 to 48 kSamples/sec. Normally, the delay is easiest to apply at the IF rate, where the stereo path's band-pass filters also operate. For a 101 tap filter, the required delay is $(N-1)/2 = 50$ samples. If that delay block is moved after the decimator, the sample rate is lower, so the equivalent delay must be scaled. Using the same time delay, 50 samples at 240 kSamples/sec becomes $50 \times \frac{48k}{240k} = 10$ samples at 48 kSamples/sec. This would result in the shift register state shrinking from 50 elements to 10 elements, which would be the new number of delayed samples for the all-pass filter. So yes, the delay block can be moved, but only if the delay is converted properly so the delay block on the mono path matches the cumulative delay caused by the two bandpass filters in the stereo path before the left and right channels are recombined. Otherwise, there would be phase misalignment between the mono and stereo paths. The same idea applies in the other modes as well, except the delay must be scaled according to that mode's own decimation or resampling ratio.

Analysis and Measurements

MAC Operations [General Functions]

blockConvolve_DecimateFast - Each output sample evaluates all N_{taps} in the innerloop - the outer loop steps by the decimation factor, computing the filter output for samples that exist post-decimation [101 MACs].

blockConvolve_ResampleFast - The filter has total taps equal to the N_{taps} multiplied by the upsampling factor - however, the inner loop steps by the upsampling factor. Thus, there are only roughly N_{taps} per output sample [101 MACs].

fmDemodNoArctan - We notice that per IF sample, there are 4 multiplications and 1 division. For simplicity, we assume that the division counts as a multiplication - thus, there are a total of 5 multiplications.

pllBlock - We notice that there are a total of 6 multiplications, one use of inverse tan and sin, and finally two uses of the cosine function.

pllBlockIQ - We notice that there are a total of 7 arithmetic multiplications, one use of inverse tan, two uses of sin as well as cosine.

In the table below is summarized the total amount of MAC operations per sample for the mono mode. Please note that some final values are rounded.

Block	Formula	Mode 0	Mode 1	Mode 2	Mode 3
RF LPF (I+Q)	$2 * 101 * R$ [$R = D/U$]	1010	1616	1099	1649
FM Demod	$5 * R$	25	40	27	41
AudioLPF	101[constant]	101	101	101	101
Totals	N/A	1136	1757	1227	1791

Non-Linear Operations - The C++ implementation replaces atan2 with a single divide (FmDemodNoArctan). Per IF sample, the number of division operations depends on R - for modes 0,1, and 2, it is approximately 5, 8, and 5.

In the table below is summarized the total amount of MAC operations per sample for the stereo mode. Please note that some final values are rounded.

Block	Formula	Mode 0	Mode 1	Mode 2	Mode 3
Mono Path	Totals from above	1136	1757	1227	1791
Pilot BPF	$101 * R$	505	808	550	824
PLL (MUL only)	$6 * R$	30	48	33	49
Stereo BPF	$101 * R$	505	808	550	824

Mixer	1 * R	5	8	5	8
Stereo LPF	101	101	101	101	101
L/R Combine	2	2	2	2	2
Totals	N/A	2284	3532	2468	3599

Non-Linear Operations - The stereo PLL performs 1 atan2, 1 sin, and 2 cos per IF sample (detailed above). Per audio sample this becomes 4 * R (20 operations for modes 0 and 2, and 32 operations for modes 1 and 3).

In the table below is summarized the total amount of MAC operations per sample for the stereo mode. Please note that some final values are rounded.]

Block	Formula	Mode 0	Mode 2
RDS BPF (54 - 60 khz)	101 * 2S [S = IF_Fs /2375]	20402	20402
Squaring Non-Linearity	1 * 2S	202	202
114 Khz BPF	101 * 2S	20402	20402
PLL IQ (MUL only)	7 * 2S	1414	1414
Allpass delay	2S copies	202	202
Mixer (I+Q)	2 * 2S	404	404
Resample (I + Q)	2 * 101 * 2 * SPS	16968	8484
RRC (I+Q)	2 * 101 * 2 * SPS	16968	8484
Totals	N/A	76,962	59,994

Non-Linear Operations - The RDS PLL performs 1 atan2, 2 sin, and 2 cos operations per IF sample (detailed above). Per audio sample this becomes 4 * R (20 operations for modes 0 and 2, and 32 operations for modes 1 and 3). Per bit, this becomes 5 * 2S = 5 * 202 = 1010 non-linear operations.

Block	MACs/block	600 MHz	900 MHz	1.2 GHz	1.5 GHz
RF LPF I	969,600				
RF LPF Q	969,600				
FM Demod	48,000				

Mono LPF	193,920				
Pilot BPF	969,600				
PLL	57,600 + 38,400 trig				
Stereo BPF	969,600				
Mixer	9,600				
Stereo LPF	193,920				
L/R Combine	3,840				
Total	4,385,280 + trig				

Block	MACs/block	600 MHz	900 MHz	1.2 GHz	1.5 GHz
RF LPF I	969,600				
RF LPF Q	969,600				
FM Demod	48,000				
Mono LPF	193,920				
Pilot BPF	969,600				
PLL	57,600 + 38,400 trig				
Stereo BPF	969,600				
Mixer	9,600				
Stereo LPF	193,920				
L/R Combine	3,840				
Total	4,385,280 + trig				

Improvement Proposals

Graphical User Interface for Station Interaction

The current system sends audio to stdout and prints RDS metadata to stderr. This setup means users cannot easily tune stations, view station names, or read radio text without checking a terminal. A simple way to improve this would be to create a graphical frontend that shows this information clearly.

The proposed GUI would display the current station frequency, the PS name and Radio Text decoded from RDS, the Programme Type, and a signal strength indicator based on the power of the FM demodulated signal. A frequency dial or slider would let users retune without restarting the program by changing the frequency argument sent to rtl_sdr while it runs. The RDS radio text display would refresh in real time as new segments come in from the decoder.

This change targets user experience directly because it eliminates the need for any terminal interaction. It presents the radio as a consumer-friendly application instead of just a command-line tool. On the Raspberry Pi 4, using a lightweight GUI toolkit like Tkinter in Python or a basic web interface served locally through a browser would be practical options that do not put much strain on the signal processing pipeline.

The Raspberry Pi 4 is a capable single-board computer, but its ARM Cortex-A72 processor has a relatively low clock speed and limited memory bandwidth compared to modern desktop or server hardware. The project currently needs to reduce the filter tap count as the CPU frequency decreases, which lowers signal quality to meet real-time requirements. Using more powerful hardware would allow for higher tap counts at all frequencies, which would improve filter stopband rejection and transition band sharpness in all modes.

Upgraded Hardware for Improved Real-Time Headroom

A practical upgrade would be to switch to a Raspberry Pi 5. It uses the ARM Cortex-A76 processor, which has much better single-core performance and improved memory bandwidth. The same software would work without changes, and the extra capacity would allow for 101 taps to be maintained at lower clock frequencies. Alternatively, it would enable more aggressive filtering for better channel selectivity. Another option would be to use a multi-core server or desktop processor. In that case, each processing thread could be assigned to a dedicated physical core instead of sharing cores with the operating system and other processes. This setup would reduce scheduling delays and make real-time performance more predictable.

Project Activity

Week #	Ayaan	Azaan	Haseeb	Taimur
1	-	-	-	-
2	Lab 3 Review and Planning	Lab 3 Review and Planning	Lab 3 Review and Planning	Lab 3 Review and Planning
3	Mono Mode 0 and	Mono Mode 0 and	Mono Mode 0 and	Mono Mode 0 and

	Stereo Research	Stereo Research	Stereo Research	Stereo Research
4	Mono Modes 1, 2, 3 Modelling and CPP integration	Mono Modes 1, 2, 3 Modelling and CPP integration	Stereo Modelling (Lead)	Stereo Modelling (Lead)
5	Mono Code Optimization (Lead)	Mono Code Optimization	Stereo C++ Implementation and Optimization	Stereo C++ Implementation and Optimization
6	RDS Phase 1 Modelling	RDS Phase 1 Modelling	RDS Modelling (Channel Extraction + Carrier Recovery)	RDS Modelling (Channel Extraction + Carrier Recovery)
7	RDS Demodulation C++	RDS Data Processing C++	Multithreading Logic & Integration	Multithreading Logic & Integration
8	RDS Finalization & Mono Reuse Optimization	RDS Finalization & Testing	Threading Debugging & Small Optimizations	Threading Debugging & Small Optimizations

The implementation was correct and fast enough to run in real time on a resource-constrained embedded platform. A correct design that ran too slowly was not useful, and a fast design that produced incorrect output was equally unacceptable. This made the project challenging because it required careful attention to both DSP accuracy and computational efficiency at every stage.

The difficulty was increased by the strong dependence between blocks in the signal-processing chain. The stereo path required proper carrier recovery and timing alignment with the mono branch, while the RDS path depended on accurate filtering, delay matching, and continuous PLL state across block boundaries. Errors in these areas were often difficult to detect because the system could still produce output that seemed reasonable while being technically incorrect. For this reason, the staged workflow of validating Python models first, then porting and optimizing them in C++, was essential to making the full system reliable.

Conclusion

Overall, this project was a really good learning experience because it forced us to go beyond just understanding the theory and actually make a full system work. As a group, we saw that even if one block makes sense on its own, that does not mean the whole SDR will work once everything is connected together. We had to keep going back and forth between modelling, implementation,

testing, and debugging, especially when moving from Python into C++. The project also helped us build a much better understanding of topics like filtering, decimation, PLLs, resampling, stereo recovery, and RDS decoding, but in a more practical way than in the labs. More than anything, it showed us that building a real system is a lot about patience, iteration, and figuring out whether a problem comes from the DSP idea itself or from how it was implemented in code.

References

McMaster University, Department of Computing and Software. (2026). 3dy4-2026-project-overview [Course slides].

McMaster University, Department of Computing and Software. (2026). 3dy4-project [Project specification].

McMaster University, Department of Computing and Software. (2026). 3dy4-lab4 [Lab manual].

McMaster University, Department of Computing and Software. (2026). 3DY4-2026-project-report [Project report and presentation guidelines].

McMaster University, Department of Computing and Software. (2026). 3dy4-2026-rds [Course slides].

McMaster University, Department of Computing and Software. (2026). 3dy4-2026-stereo [Course slides].